

Set 2 - Classes and Methods

Q1.

Create a Java class named Playlist with a method addSong() that prints 'Song added to playlist' and another method checkSongCount() that prints the total number of songs added.

Answer:

```
public class Playlist {
    private int songCount = 0;

    public void addSong() {
        songCount++;
        System.out.println("Song added to playlist");
    }

    public void checkSongCount() {
        System.out.println("Total songs in playlist: " + songCount);
    }

    public static void main(String[] args) {
        Playlist myPlaylist = new Playlist();
        myPlaylist.addSong();
        myPlaylist.addSong();
        myPlaylist.addSong();
        myPlaylist.checkSongCount();
    }
}

/* Sample Output:
Song added to playlist
Song added to playlist
Song added to playlist
Total songs in playlist: 3
*/
```

Q2.

Define a class called MovieTicket with an instance variable price and a static variable totalTickets. Write methods bookTicket(int amount) to increase totalTickets and checkTotalTickets() to print the current total. Hint: Use the static keyword for totalTickets so it is shared across all MovieTicket objects.

Answer:

```
public class MovieTicket {
    private int price; // instance variable - unique per object
    private static int totalTickets = 0; // static variable - shared by all objects

    public void bookTicket(int amount) {
        this.price = amount;
        totalTickets++;
        System.out.println("Ticket booked for Rs. " + price);
    }

    public static void checkTotalTickets() {
        System.out.println("Total tickets booked so far: " + totalTickets);
    }
}
```

```

    public static void main(String[] args) {
        MovieTicket t1 = new MovieTicket();
        MovieTicket t2 = new MovieTicket();

        t1.bookTicket(250);
        t2.bookTicket(300);

        MovieTicket.checkTotalTickets(); // shared across all objects
    }
}

/* Sample Output:
Ticket booked for Rs. 250
Ticket booked for Rs. 300
Total tickets booked so far: 2
*/

```

Q3.

Write a class called FoodOrder with methods addItem(String itemName) and getOrderSummary() that keeps track of items added and displays the full order when called.

Answer:

```

import java.util.ArrayList;

public class FoodOrder {
    private ArrayList<String> items = new ArrayList<>();

    public void addItem(String itemName) {
        items.add(itemName);
        System.out.println(itemName + " added to your order.");
    }

    public void getOrderSummary() {
        System.out.println("---- Order Summary ----");
        if (items.isEmpty()) {
            System.out.println("No items in the order yet.");
            return;
        }
        for (String item : items) {
            System.out.println("- " + item);
        }
        System.out.println("Total items: " + items.size());
    }

    public static void main(String[] args) {
        FoodOrder order = new FoodOrder();
        order.addItem("Paneer Butter Masala");
        order.addItem("Butter Naan");
        order.addItem("Cold Coffee");
        order.getOrderSummary();
    }
}

/* Sample Output:
Paneer Butter Masala added to your order.
Butter Naan added to your order.
Cold Coffee added to your order.
---- Order Summary ----
- Paneer Butter Masala
- Butter Naan

```

```
- Cold Coffee
Total items: 3
*/
```

Q4.

Create a class called InstaProfile with instance variables username and followers, and a method increaseFollowers(int count) that adds to followers and prints the updated count.

Answer:

```
public class InstaProfile {
    private String username;
    private int followers;

    public InstaProfile(String username, int followers) {
        this.username = username;
        this.followers = followers;
    }

    public void increaseFollowers(int count) {
        followers += count;
        System.out.println(username + " now has " + followers + " followers.");
    }

    public static void main(String[] args) {
        InstaProfile profile = new InstaProfile("tech_with_ravi", 1500);
        profile.increaseFollowers(120);
        profile.increaseFollowers(45);
    }
}

/* Sample Output:
tech_with_ravi now has 1620 followers.
tech_with_ravi now has 1665 followers.
*/
```

Q5.

Relate the concept of classes and methods to a real app you use: Pick any one (like Zomato, Flipkart, Instagram) and list 3 possible classes (as if you were coding the app) and for each, write 2 method names that would perform CRUD-like operations.

Answer:

```
class Restaurant {
    void addRestaurant() { System.out.println("New restaurant added (Create)"); }
    void updateMenu() { System.out.println("Menu updated (Update)"); }
}

class Order {
    void placeOrder() { System.out.println("Order placed (Create)"); }
    void cancelOrder() { System.out.println("Order cancelled (Delete)"); }
}

class UserProfile {
    void updateAddress() { System.out.println("Address updated (Update)"); }
    void deleteAccount() { System.out.println("Account deleted (Delete)"); }
}
```

App chosen: Zomato

1. Class: Restaurant - `addRestaurant()` -> Create a new restaurant listing - `updateMenu()` -> Update/modify the restaurant's menu items
2. Class: Order - `placeOrder()` -> Create a new food order - `cancelOrder()` -> Delete/cancel an existing order
3. Class: UserProfile - `updateAddress()` -> Update the user's saved delivery address - `deleteAccount()` -> Delete the user's account/profile

Just as a database table has Create, Read, Update, Delete operations, each class in an app groups together the methods that manage its own piece of data - the Restaurant class manages restaurant data, the Order class manages order data, and so on. This keeps the code organized and each class responsible for one part of the app.